

Tutorial 04

Lecture reviews.

Design Techniques

1. **Express all constraints** (you can express!).
2. Use and do not change terminology and class structure of the application domain.
3. Keep it simple.
 1. Avoid defining entity types that do not serve any purpose.
 2. **Don't use an entity set when an attribute will do.** Choose an entity set if it helps expressing constraints; otherwise, use an attribute.
4. Avoid redundancy (but derived attributes are okay)!
5. Limit the use of weak entity sets.

Weak Entity Sets

- Database design beginners often doubt that any attribute could be a key.
- Once there is no solution for a key, they make all entity sets weak, supported by all other entity sets to which they are linked.
- In fact, we can artificially create unique IDs for entity sets.
- We use weak entity sets **only if there is no capability of creating unique IDs.**

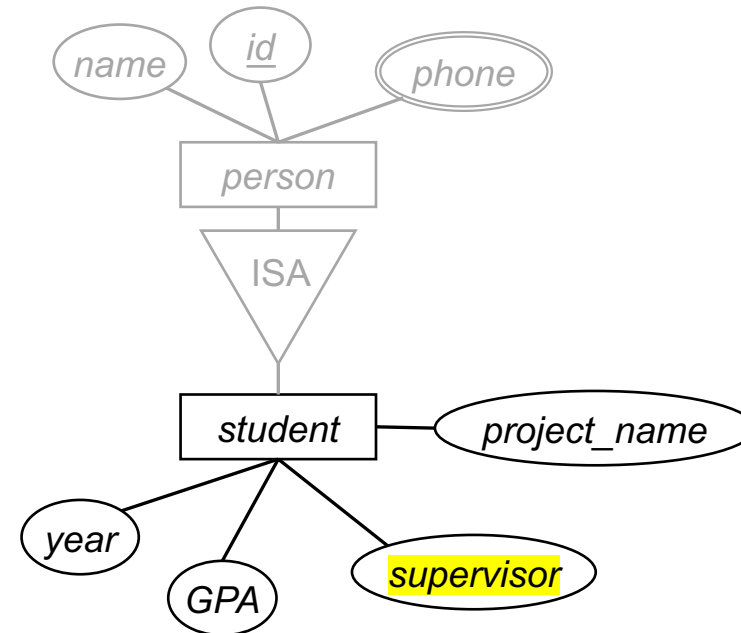
Redundancy Elimination

- Suppose we want to model that “each student is doing a group project with a supervisor”.
- What is the problem of this design?

- Remember, this is a group project. Multiple students can work on the same project.
- It is possible that

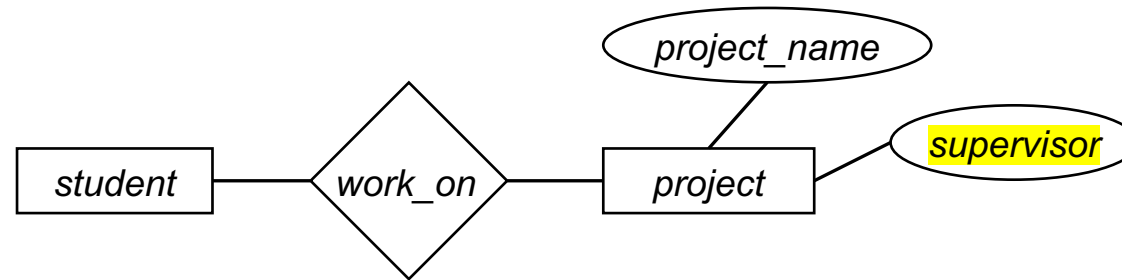
students = {
 (..., Alice, 3D Modeling, Amy),
 (..., Bob, 3D Modeling, Amy),
 (..., Car, 3D Modeling, Amy),
 ... } student project supervisor

- It creates **redundancies**.



Redundancy Elimination

- A better design can be



Exercises– Redundancy Elimination

- Consider the UIC database that we have been discussed for previous lectures. (The ER diagram is given on the next page.)
 - Why the entity set *transcript* is useless?
 - Transcript is useless because
 - 1) **gained units** depend on the course enrollment, which can be calculated;
 - 2) **transcript id** does not present any information.

Lab.

review

Aggregation

- Suppose that we want to count the number of different languages.
- The **aggregation function** **COUNT** can be used.

```
SELECT COUNT(*) FROM language
```

- “Aggregation” means **performing some operations on a group of objects.**

GROUP BY

- For example, find the **number of films for each language.** The resulting table has two columns, one for languages, one for the number of films.
- Only those films of the same language are in the same group (different language different group).
- Then, count the number within each group.
- To achieve the grouping, we use **GROUP BY**.

```
SELECT language_id, count(film_id)
FROM film JOIN language USING(language_id)
GROUP BY (language_id)
```

- To indicate the result of aggregation of each group, **group by attributes** are usually selected.

GROUP BY

- The GROUP BY clause contains a set of attributes.
- Tuples with the same value of **all** group by attribute(s) are in the same group.
- Be careful when the group by attribute is **not a key**.
- Compare the two queries and tell the difference.

```
SELECT first_name, COUNT(film_id)
FROM actor JOIN film_actor USING(actor_id)
GROUP BY first_name
```

```
SELECT actor_id, first_name, COUNT(film_id)
FROM actor JOIN film_actor USING(actor_id)
GROUP BY actor_id
```

HAVING

- Sometimes we need condition checking before and after aggregation functions.
- For example, show the actors' names and the number of Sci-Fi movies played by him/her if the number of Sci-Fi movies is more than 3.

```
SELECT actor_id,first_name,last_name,COUNT(film_id)
```

```
FROM actor JOIN film_actor USING (actor_id)  
      JOIN film USING(film_id)  
      JOIN film_category USING(film_id)  
      JOIN category USING(category_id)
```

```
WHERE category.name='sci-fi'
```

```
GROUP BY actor_id
```

```
HAVING COUNT(film_id)>3;
```

The query is executed in this sequence.

1. Combine the tables in FROM clause
2. Check the condition in WHERE clause
3. Group tuples
4. Aggregation function
5. Check the condition in HAVING clause
6. Output the selected attributes

Aggregation

- Other than COUNT, there are more aggregate functions for different purposes.
 - **MAX**: find the maximum value
 - **MIN**: find the minimum value
 - **AVG**: calculate the average value
 - **SUM**: sum up the values

Distinct

- We only assume that every country has a city. But a country may have multiple cities.
- Those countries will be over counted.
- We can use **DISTINCT** to remove duplications.

```
SELECT COUNT(DISTINCT country_id)FROM city
```

1. Calculate the average rental duration among all films.

```
SELECT film_id, title, AVG(rental_duration)  
FROM film
```

2. Calculate the number of films rented by Norman Carrier.

```
SELECT COUNT(DISTINCT film_id)
FROM customer JOIN rental USING(customer_id) JOIN inventory
USING(inventory_id)
WHERE first_name='Norman' AND last_name='Carrier';
```

3. Calculate the number of films rented by each customer.

```
SELECT customer_id, first_name, last_name, COUNT(DISTINCT  
film_id)  
FROM customer JOIN rental USING(customer_id) JOIN inventory  
USING(inventory_id)  
GROUP BY customer_id;
```

4. Calculate the number of films rented by each group of customers from the same country.

```
SELECT country_id, country, COUNT(DISTINCT film_id)
FROM customer JOIN rental USING(customer_id) JOIN inventory
USING(inventory_id) JOIN address USING(address_id) JOIN city
USING(city_id) JOIN country USING(country_id)
GROUP BY country_id;
```

5. Calculate the number of horror films rented by each group of customers from the same country.

```
SELECT country_id, country, COUNT(DISTINCT film_id)
FROM customer JOIN rental USING(customer_id) JOIN inventory
USING(inventory_id) JOIN address USING(address_id) JOIN city
USING(city_id) JOIN country USING(country_id) JOIN film_category
USING(film_id) JOIN category USING(category_id)
WHERE category.name='horror'
GROUP BY country_id;
```

6. Find the countries which have some customers who have rented more than 4 horror films.

```
SELECT country_id, country
FROM customer JOIN rental USING(customer_id) JOIN inventory
USING(inventory_id) JOIN address USING(address_id) JOIN city
USING(city_id) JOIN country USING(country_id) JOIN film_category
USING(film_id) JOIN category USING(category_id)
WHERE category.name='horror'
GROUP BY customer_id
HAVING COUNT(DISTINCT film_id)>4;
```

Q&A